AGH UNIVERSITY OF KRAKOW

Numerical Methods in Physics

Lab 1 - Vector and Matrix Operations

From C++ loops to CBLAS

Hamzeh Khanpour | AGH University of Krakow | 9 October 2026

Adapted for teaching from the practical assignment by Tomasz Chwiej (3 March 2026).

By the end of this laboratory, you should be able to...

1. Implement scalar products and Euclidean vector norms in `C++17`.
2. Store matrices in one-dimensional memory arrays and understand **LDA**.
3. Compute $\boldsymbol{x}^{(m)} = A^m \boldsymbol{x}^{(0)}$ and save reproducible data.
4. Implement $C = A^2$ by nested loops and by `cblas_dgemm()`.
5. Validate numerical outputs and compare real measured runtimes.

Core lesson

A numerical formula is only useful when indices, storage layout and numerical verification all agree.

Today's roadmap

Minutes	Task	Checkpoint
0–25	Vector products, norms, C++ compile test	Mini examples
25–40	Row-/column-major storage, LDA	Indexing quiz
40–60	Matrix-vector code, correctness tests	2×2 result
60–85	Full 2000×2000, $m = 30$, $\gamma = 2, 3$	Two CSV files
85–105	Manual matrix multiplication	2×2 validation
105–120	CBLAS and timing, if installed	Numeric comparison
120–135	Plots, report, packaging, questions	Submission plan

What the assignment requires?

Experiment A: repeated Ax

$A_{ij} = \dfrac{1}{|i-j|^4 + \gamma}$, $n = 2000$. $\boldsymbol{x}^{(0)} = \mathbf{1}$, $m = 1, \ldots, 30$; run $\gamma = 2$ and 3. Save m and $\|\boldsymbol{x}^{(m)}\|_2$ after each step.

Experiment B: A^2

Compute $C = A\,A$ twice:

- Manual triple loop.
- BLAS3 `cblas_dgemm`.

Compare accuracy and timings.

Homework: PDF report

Show a **separate norm plot** for each γ, compare measured timings, and describe observed behavior.

Translate mathematics into C++ operations

Mathematics	C++ implementation
$\boldsymbol{x} \cdot \boldsymbol{y} = \sum_i x_i y_i$	Accumulation inside one loop
$\|\boldsymbol{x}\|_2 = \sqrt{\sum_i x_i^2}$	Sum-of-squares then `std::sqrt`
$y_i = \sum_j A_{ij} x_j$	Outer i, inner j
$C_{ik} = \sum_j A_{ij} B_{jk}$	Three loops over i, k, j
$A_{ij} \leftrightarrow A[i \cdot \mathrm{LDA} + j]$	Flat memory (row-major)

Programming discipline

Start from a small example whose result can be calculated by hand. Only then run at $n = 2000$.

Scalar product: one formula, one loop

$$\boxed{\boldsymbol{x}^T \boldsymbol{y} = \sum_{i=0}^{n-1} x_i y_i}$$

Worked example

$\boldsymbol{x} = (1, 2, 3)$, $\boldsymbol{y} = (4, -1, 2)$.

$\boldsymbol{x}^T \boldsymbol{y} = 1(4) + 2(-1) + 3(2) = \boxed{8}$.

C++ idea

```
double s = 0.0;
for (i = 0; i < n; ++i)
  s += x[i] * y[i];
```

Euclidean norm and numerical accuracy

$$\|\boldsymbol{x}\|_2 = \sqrt{\boldsymbol{x}^T \boldsymbol{x}} = \sqrt{\sum_{i=0}^{n-1} x_i^2}$$

Worked example

$\boldsymbol{x} = (3, 4)$ gives $\|\boldsymbol{x}\|_2 = \sqrt{9 + 16} = \boxed{5}$.

At $m = 0$, $\boldsymbol{x}^{(0)} = \mathbf{1}$ with $n = 2000$:
$\|\boldsymbol{x}^{(0)}\|_2 = \sqrt{2000} \approx 44.7214$.

Numerical issue

Straight $\sqrt{\sum x_i^2}$ can overflow for very large entries.

Row-major layout: the last index changes fastest

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

1	2	3
4	5	6

Flat C++ array

$$\underbrace{[1, 2, 3,\ 4, 5, 6]}_{\text{indices } 0 \ldots 5}$$

$\text{offset}(i, j) = i \times \text{LDA} + j$

For $\text{LDA} = 3$, $A_{1,2} \to a[5] = 6$.

Column-major layout: the first index changes fastest

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

1	4	2	5	3	6

One-dimensional layout: [1,4,2,5,3,6]

Fortran / column-major convention

$\text{offset}(i, j) = i + j \times \text{LDA}$

For $\text{LDA} = 2$ and $A_{1,2} = 6$:

$$\text{offset} = 1 + 2(2) = \boxed{5}.$$

Practical rule

Always match the BLAS layout flag to how your array is actually stored.

Leading dimension (LDA) is not always the matrix width

Row-major, logical 2×3, physical **LDA=5**:

1	2	3	–	–
4	5	6	–	–

Correct index examples

$A_{0,2} \to a[0(5) + 2] = a[2]$

$A_{1,0} \to a[1(5) + 0] = a[5]$

$A_{1,2} \to a[1(5) + 2] = a[7]$.

C++ micro-exercise: create and inspect row-major storage

```cpp
const int rows=2, cols=3, lda=5;
std::vector<double> a(rows * lda, -99.0);
for (int i=0; i<rows; ++i) {
    for (int j=0; j<cols; ++j) {
        a[i * lda + j] = i * cols + j + 1;
    }
}
// Flat memory: 1 2 3 -99 -99 4 5 6 -99 -99
```

Try it now

Compile `examples/01_storage_layout.cpp`. Explain why $a[5] = 4$ while $a[3] = -99$.

Indexing quiz: derive addresses without guessing

For a **row-major** 4×3 matrix with $\mathrm{LDA} = 5$:

1. What is the location of $A_{2,1}$? $2 \cdot 5 + 1 = \boxed{11}$.
2. What is the location of $A_{3,2}$? $3 \cdot 5 + 2 = \boxed{17}$.
3. What is the physical storage size? $4 \cdot 5 = \boxed{20}$ doubles.
4. Which logical row elements are contiguous? $A_{i,0}, A_{i,1}, A_{i,2}$.

Matrix-vector multiplication: mathematical definition

$$y_i = \sum_{j=0}^{n-1} A_{ij} x_j, \quad i = 0, \ldots, n-1$$

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x^{(0)} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$x^{(1)} = Ax^{(0)} = \begin{pmatrix} 3 \\ 3 \end{pmatrix},$$

$$x^{(2)} = Ax^{(1)} = \begin{pmatrix} 9 \\ 9 \end{pmatrix}.$$

Verify the norm

$\|x^{(1)}\|_2 = 3\sqrt{2}$ and $\|x^{(2)}\|_2 = 9\sqrt{2}$.

C++ reference implementation: $y = Ax$

```cpp
void matvec(const std::vector<double>& A,
            const std::vector<double>& x,
            std::vector<double>& y, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {
        double s = 0.0;
        for (std::size_t j = 0; j < n; ++j) {
            s += A[i*n + j] * x[j];
        }
        y[i] = s;
    }
}
```

Three bugs that silently spoil Ax

Incorrect

```
y[i] = A[i*n+j]*x[j];
```

⇒ Each new term **overwrites** the sum.

Incorrect

```
A[j*n+i]
```

⇒ Uses the transposed address for row-major matrices.

Correct

```
s += A[i*n+j]*x[j];
```

`y[i] = s;` outside inner loop.

Incorrect

```
matvec(A, x, x, n);
```

⇒ Input overwritten while still being read.

Repeated multiplication: use two vector buffers

$$\boldsymbol{x}^{(m)} = A\boldsymbol{x}^{(m-1)}, \qquad m = 1, \ldots, 30$$

```
std::vector<double> x(n, 1.0), next(n, 0.0);
for (int m = 1; m <= 30; ++m) {
    matvec(A, x, next, n); // next = A*x
    x.swap(next);          // x becomes the new vector
    const double norm = euclidean_norm(x);
    csv << m << "," << norm << "\n";
}
```

Advantage

Swapping vectors avoids allocating a new n-vector at every iteration and prevents in-place aliasing.

The official numerical matrix is symmetric and positive

$$A_{ij} = \frac{1}{|i-j|^4 + \gamma}, \qquad n = 2000, \quad \gamma \in \{2, 3\}$$

For $\gamma = 2$

$A_{ii} = 1/2$; $A_{i,i\pm1} = 1/3$; $A_{i,i\pm2} = 1/18$.
Nearby entries contribute substantially.

For $\gamma = 3$

$A_{ii} = 1/3$; $A_{i,i\pm1} = 1/4$; $A_{i,i\pm2} = 1/19$.
The entries are smaller throughout.

Why do the norms grow for $\gamma = 2$ and shrink for $\gamma = 3$?

$$x^{(m)} = A^m x^{(0)}, \qquad \|x^{(m)}\|_2 \sim \text{const} \cdot \rho(A)^m$$

$\gamma = 2$: amplification

For $n = 2000$, the dominant eigenvalue is approximately $\rho(A) \approx \boxed{1.3167} > 1$.
The vector norm grows over iterations.

$\gamma = 3$: attenuation

For $n = 2000$, the dominant eigenvalue is approximately $\rho(A) \approx \boxed{0.9772} < 1$.
The vector norm decays over iterations.

Interpretation exercise: track the norm ratios

Define $r_m = \|x^{(m)}\|_2 / \|x^{(m-1)}\|_2$.

Discuss in pairs

1. What does $r_m > 1$ mean?
2. What does $r_m < 1$ mean?
3. Why does r_m approach a constant?

Expected explanation

The leading positive eigenmode increasingly dominates.
Therefore $r_m \to \rho(A)$, when that mode has nonzero weight.

Save results in an analysis-friendly CSV file

```
std::ofstream csv("results/norm_gamma2.csv");
csv << "iteration,norm\n" << std::setprecision(17);
for (int m=1; m<=30; ++m) {
    // update x with A*x, then compute norm
    csv << m << ',' << euclidean_norm(x) << '\n';
}
```

Example rows ($\gamma = 2$)

```
iteration,norm
1,58.86673138
10,699.5854296
30,171507.0793
```

Commands

```
make basic
./bin/lab01_norms
python3 plotting/plot_norms.py
results
```

Matrix-matrix multiplication: the second required task

$$\boxed{C = AB, \quad C_{ik} = \sum_{j=0}^{n-1} A_{ij} B_{jk}}$$

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \qquad AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Verify one element

$C_{1,0} = A_{1,0}B_{0,0} + A_{1,1}B_{1,0} = 3(5) + 4(7) = \boxed{43}$.

Reference triple-loop: row-major $C = A\,A$

```
for (int i=0; i<n; ++i) {
    for (int k=0; k<n; ++k) {
        double sum = 0.0;
        for (int j=0; j<n; ++j) {
            sum += A[i*n+j] * A[j*n+k];
        }
        C[i*n+k] = sum;
    }
}
```

Cost

Approximately n^3 multiplications and n^3 additions: for $n = 2000$, about 16×10^9 floating operations for one $C = A^2$ computation.

BLAS: Basic Linear Algebra Subprograms

Level	Typical operation	C interface example
BLAS1	Vectors: dot product, norm	`cblas_ddot`, `cblas_dnrm2`
BLAS2	Matrix-vector multiplication	`cblas_dgemv`
BLAS3	Matrix-matrix multiplication	`cblas_dgemm`

Why BLAS3?

A high-quality GEMM implementation reuses cache data, vectorizes arithmetic and may use several CPU threads. The observed speed depends on the installed BLAS implementation.

Understanding the `cblas_dgemm` API

$$C \leftarrow \alpha\, \mathrm{op}(A)\, \mathrm{op}(B) + \beta C$$

Official task arguments

- Layout: `CblasRowMajor`
- Transposes: `CblasNoTrans` twice
- $m = n = k = 2000$
- $\alpha = 1$, $\beta = 0$
- `lda=ldb=ldc=2000`

Meaning of $\beta = 0$

Ignore the previous contents of output matrix C.

Critical: if you accidentally use $\beta = 1$, C is accumulated instead of replaced.

The complete BLAS3 call for $C = A^2$

```
#include <cblas.h>
// A and C: std::vector<double>, row-major.
cblas_dgemm(CblasRowMajor,
            CblasNoTrans, CblasNoTrans,
            n, n, n,          // M, N, K
            1.0, A.data(), n,
                 A.data(), n,
            0.0, C.data(), n);
```

Compilation (where CBLAS is actually installed)

```
g++ -O2 -std=c++17 source.cpp -o program -lblas -lm
```

Numerical verification: manual vs BLAS

1. Test both methods on a 2×2 example with $C = \left(\begin{smallmatrix} 19 & 22 \\ 43 & 50 \end{smallmatrix}\right)$.
2. Compute C_{manual} and C_{BLAS} using exactly the same matrix A.
3. Check a **relative Frobenius error**:

$$\varepsilon_F = \frac{\|C_{\text{manual}} - C_{\text{BLAS}}\|_F}{\|C_{\text{manual}}\|_F}$$

Interpretation

For a correctly implemented problem in double precision, differences should be small. Do *not* require bit-for-bit identity across all BLAS implementations: floating-point summation order may differ.

Computational cost: why performance matters

	Formula	**At $n = 2000$**
One matrix-vector	$O(n^2)$	$4 \cdot 10^6$ terms
30 matrix-vector	$30n^2$	$1.2 \cdot 10^8$ terms
One matrix-matrix	$O(n^3)$	$8 \cdot 10^9$ terms
One double matrix	$8n^2$ bytes	$32,000,000$ bytes
Three matrices	$3 \cdot 8n^2$ bytes	$96,000,000$ bytes

Key distinction

Large numbers of operations do not directly predict elapsed time: memory access patterns, CPU, compiler optimization and BLAS provider matter.

Measure timings carefully

Modern C++ clock

```
auto t0 = steady_clock::now();
// multiply matrices
auto t1 = steady_clock::now();
duration<double>(t1-t0).count()
```

Benchmark protocol

1. Time 3 or more runs.
2. Compare equal matrix sizes.
3. Report optimization and CPU.
4. Record BLAS provider/thread count.
5. Verify output before claiming speedup.

Check the computing environment first

```
which g++
g++ --version
make basic
make test
make check-env
# If CBLAS is available:
make blas
make test-blas
```

Important distinction

The coordinator's page contains Taurus BLAS instructions, but installations can change. A successful **compile/link probe** is stronger evidence than an old webpage claim. The vector experiment can be completed without BLAS.

Optional extension: does C++ loop order matter?

Official reference loops

```
for i
 for k
  for j
```

- Reads A row-wise.
- Reads B down a column.

Cache-friendly extension

```
for i
 for j
  for k
```

- Reuses A_{ij}.
- Reads B across a row.

Research question

With $-O2$ held constant, do the two loop orders take the same time at $n = 256$ or 512? Record your observations.

Optional extension: BLAS1 and BLAS2

Math	C interface
$s = x^T y$	`cblas_ddot(n,x,1,y,1)`
$s = \|x\|_2$	`cblas_dnrm2(n,x,1)`
$y \leftarrow Ax$	`cblas_dgemv(CblasRowMajor,...)`

Challenge

Compare a custom matvec with `cblas_dgemv` for $n = 512$ and verify that the vectors agree within floating-point tolerance.

Optional extension: locate the growth/decay transition

1. Estimate $\rho(A)$ by repeated multiplication and norm normalization.
2. Try $\gamma = 2$, 2.5, and 3.
3. Identify the approximate γ where $\rho(A) = 1$.

Prediction

Increasing γ reduces every entry of the strictly positive A; its leading eigenvalue decreases.

Method

Use `examples/05_gamma_exploration.cpp`. Check estimates using more power iterations.

Student checkpoint: can you demonstrate all six?

1. Your code compiles on the lab machine.
2. Your 2×2 manual calculations match C++ output.
3. You can explain row-major `A[i*LDA+j]` indexing.
4. You produced 30 norms for **each** gamma with $n = 2000$.
5. You compared manual A^2 with `cblas_dgemm` (where available), including correctness checks.
6. Your source, numerical data, figures and report can be reproduced by another student/instructor.

The required report: scientific evidence, not screenshots

Minimum numerical figures

- Plot 1: m vs $\|x^{(m)}\|_2$ for $\gamma = 2$.
- Plot 2: same for $\gamma = 3$.
- Clearly label axes and the gamma value.
- Measured timing comparison for two A^2 methods.

Suggested report structure

1. Method (about 1 page).
2. Task and parameters (about half page).
3. Figures/tables with brief interpretation.
4. Conclusions (about half page).

Submission packaging and lab rules

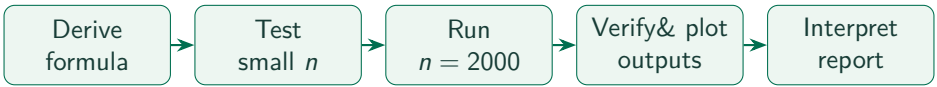
Practical archive

- Keep C++ source in one directory.
- Save both CSV norm sequences.
- Save actual benchmark measurements.
- Zip the code + numeric data.

PDF report (UPEL)

Course webpage: report due within seven days of the last laboratory session. Report and laboratory activity are separately assessed there on 0–100 scales.

A reliable scientific computing workflow



Take-home idea

Correctness first, reproducibility second, speed third. A library call is only as trustworthy as its dimensions, layout and input arrays.